Project Documentation: AI chat

Candidate Name: Abdullayev Xusan

Project Link: https://xfenixai.netlify.app/

---

1. What you built

I developed **FenixAI**, a minimalist, full-stack web application that serves as a private interface for an Artificial Intelligence. The application provides a clean, distraction-free environment where a user can input a query, and the system retrieves a coherent, context-aware response from a Large Language Model (LLM). The project bridges the gap between a simple user interface and complex AI processing.

2. Tools and technologies used

- **Frontend:** HTML5, CSS3, and JavaScript (Vanilla). I chose a minimalist approach to ensure fast loading times and a focus on functionality.
- **Backend:** Node.js with the Express.js framework to handle API routing and secure communication.
- **LLM API:** [Specify your API: e.g., Google Gemini API or Groq], which provides the "intelligence" for the chat.
- **Environment Management:** `dotenv` for managing sensitive credentials.
- **Deployment:** Netlify for the frontend and [e.g., Render/Vercel] for the backend hosting.

3. How LLM APIs work (in your own words)

An LLM API works as a bridge between my application and a massive, pre-trained brain sitting on a remote server. Instead of needing a supercomputer to run an AI locally, I send the user's text as a "prompt" to the API provider. The provider’s model processes this input—predicting the most logical sequence of words based on its training—and sends that text back to my app as a response. It essentially allows me to "plug in" high-level intelligence via a standard web request.

4. How frontend and backend communicate

The communication follows a request-response cycle. When the user clicks "Send," the frontend uses the **Fetch API** to send an asynchronous `POST` request to a specific endpoint on my Node.js backend. This request contains a JSON object with the user's message. The backend receives this, processes it, and once it gets a result from the AI, it sends a JSON response back to the frontend. The frontend then dynamically updates the DOM to show the message without refreshing the page.

5. How you handled the API key securely

Security was a top priority. I never hardcoded the API key into the frontend code, as that would make it visible to anyone inspecting the website. Instead:

1. I stored the key in a `.env` file on the backend.
2. I added `.env` to my `.gitignore` file so it was never uploaded to GitHub.
3. On the hosting platform (e.g., Render/Netlify), I manually entered the key into the **Environment Variables** settings. This ensures the key only exists in the secure server environment during runtime.

6. What prompts you used when asking AI tools for help

I used AI tools to troubleshoot specific technical hurdles. Some of the prompts included:

- *"How do I set up an Express.js server to act as a proxy for the Gemini API to avoid exposing my key?"*
- *"Write a CSS snippet for a chat bubble that expands with text and stays aligned to the right."*
- *"How can I handle error status codes (like 401 or 500) in a JavaScript fetch request to show a user-friendly message?"*

7. Problems you faced and how you solved them

- **The Problem:** I initially encountered **CORS (Cross-Origin Resource Sharing)** errors because the frontend on Netlify was trying to talk to a backend on a different domain.
- **The Solution:** I implemented the `cors` middleware in my Node.js application, specifically whitelisting my frontend URL to allow secure communication between the two.
- **The Problem:** The UI felt "frozen" while waiting for the AI to respond.
- **The Solution:** I implemented a "Loading..." state and disabled the send button during the request to provide visual feedback and prevent duplicate submissions.

8. What you learned from this project

This project deepened my understanding of the **Client-Server-API** architecture. I learned that building an AI application isn't just about the "AI" part; it’s about building a secure "pipeline" for data. I also gained practical experience in environment variable management and deploying a live, full-stack product that handles real-world API constraints.